Teach Module 38: SQL and Query Performance

This README captures the Module 38 teaching notes, exercises, and practice lab. The course document was used only to identify the module topic; the teaching content below is written independently.

Module Goal

Module 38 focuses on writing SQL that is correct, readable, and efficient for real application data.

Core topics:

- Joins
- Subqueries
- Index usage
- Selectivity
- Execution plans
- Query tuning
- SQL anti-patterns
- Performance-focused practice labs

1. Big Idea: Query Performance

SQL performance is mostly about this question:

> *How much data does the database have to inspect, join, sort, or move to answer this query?*

A query can look small but be expensive if it forces the database to scan millions of rows, sort large result sets, or repeatedly run subqueries.

Example:

```sql
SELECT *
FROM orders
WHERE customer_id = 42;
```

If `orders.customer_id` is indexed, the database can often jump directly to matching rows.

If it is not indexed, the database may need to scan the whole `orders` table.

That difference is the heart of query performance.

2. Joins

A join combines rows from two or more tables.

Example:

```sql
SELECT c.name, o.order_date, o.total
FROM customers c
JOIN orders o
  ON c.id = o.customer_id;
```

This means:

- `customers` has customer information
- `orders` has order information
- `orders.customer_id` points back to `customers.id`

The database matches rows where the IDs line up.

Common joins:

```
-- Only matching rows from both tables
INNER JOIN

-- All left-side rows, plus matches if they exist
LEFT JOIN

-- All right-side rows, plus matches if they exist
RIGHT JOIN

-- All combinations of both tables, usually dangerous
CROSS JOIN
```

Most application queries use `INNER JOIN` and `LEFT JOIN`.

INNER JOIN

Use `INNER JOIN` when both records must exist.

```
SELECT c.name, o.id AS order_id, o.total
FROM customers c
INNER JOIN orders o
  ON c.id = o.customer_id;
```

This returns only customers who have matching orders.

LEFT JOIN

Use `LEFT JOIN` when the related record may not exist.

```
SELECT c.name, o.id AS order_id
FROM customers c
LEFT JOIN orders o
  ON c.id = o.customer_id;
```

This returns all customers, even if they have no orders.

For customers without orders, the order columns will be `NULL`.

Performance idea:

> *Join columns should usually be indexed, especially foreign keys such as `orders.customer_id`.*

3. Subqueries

A subquery is a query inside another query.

Example:

```
SELECT name
FROM customers
WHERE id IN (
  SELECT customer_id
  FROM orders
  WHERE total > 1000
);
```

This finds customers who have placed expensive orders.

Subqueries are useful, but they can become slow if the database has to repeatedly evaluate them.

The same result can often be written with a join:

```
SELECT DISTINCT c.name
FROM customers c
JOIN orders o
  ON c.id = o.customer_id
WHERE o.total > 1000;
```

Neither version is automatically better. Modern database optimizers can sometimes rewrite one form into another internally.

As a developer:

- Use subqueries when they express filtering clearly.
- Use joins when you need columns from multiple tables.
- Compare execution plans when performance matters.

4. Indexes

An index is a lookup structure for a table.

Without an index, finding rows can mean scanning the table.

With an index, the database can often find rows much faster.

Example:

```
CREATE INDEX idx_orders_customer_id
ON orders(customer_id);
```

This helps queries like:

```
SELECT *
FROM orders
WHERE customer_id = 42;
```

Indexes often help with:

- `WHERE`
- `JOIN`
- `ORDER BY`
- `GROUP BY`

But indexes are not free.

They cost:

- Extra disk space
- Slower inserts
- Slower updates
- Slower deletes

Every data change may also require updating the index.

Rule of thumb:

> *Index columns that are commonly used to find, join, filter, or sort meaningful amounts of data.*

5. Selectivity

Selectivity means how much a condition narrows the result.

High selectivity:

```
WHERE email = 'aisha@example.com'
```

This probably matches one row. That makes it a strong index candidate.

Low selectivity:

```
WHERE status = 'ACTIVE'
```

If 90% of users are active, the index may not help much.

A database may decide it is cheaper to scan the table than use the index.

Good index candidates usually have many distinct values:

- Email
- Username
- Order ID
- Customer ID
- Created date
- Foreign keys

Weak index candidates often have few distinct values:

- Boolean flags
- Gender
- Status with only two or three values
- Tiny category columns

These are not always bad index choices, but they are less automatically useful.

6. Execution Plans

An execution plan shows how the database intends to run your query.

You usually inspect one with something like:

```
EXPLAIN
SELECT *
FROM orders
WHERE customer_id = 42;
```

Depending on the database, the output differs, but you are usually looking for signs like:

- Full table scan
- Index scan
- Nested loop join
- Hash join
- Sort operation
- Estimated rows
- Actual rows
- Cost

The main question:

> *Did the database scan way more rows than expected?*

If yes, the query may need a better index, better filtering, or a rewrite.

7. Common SQL Anti-Patterns

Anti-Pattern 1: `SELECT *`

Avoid this in production queries when you do not need every column.

Bad:

```
SELECT *
FROM orders;
```

Better:

```
SELECT id, customer_id, order_date, total
FROM orders;
```

Why this matters:

- Less data transferred
- Less memory used
- Clearer application mapping
- More stable when table structure changes

Anti-Pattern 2: Functions on Indexed Columns

Bad:

```sql
SELECT *
FROM customers
WHERE LOWER(email) = 'aisha@example.com';
```

Applying a function to the column may prevent normal index usage.

Better:

```sql
SELECT *
FROM customers
WHERE email = 'aisha@example.com';
```

Another good option is to store normalized email values or use a function-based index if the database supports it.

Anti-Pattern 3: Leading Wildcard Searches

Bad:

```sql
SELECT *
FROM products
WHERE name LIKE '%phone';
```

A normal index usually cannot jump efficiently into the middle of text.

Better:

```sql
SELECT *
FROM products
WHERE name LIKE 'phone%';
```

For contains-style search, consider full-text search features.

Anti-Pattern 4: Unnecessary `DISTINCT`

Bad:

```sql
SELECT DISTINCT c.*
FROM customers c
JOIN orders o
  ON c.id = o.customer_id;
```

`DISTINCT` can hide data modeling or query mistakes.

It may also force expensive sorting or hashing.

If duplicates appear, first understand why they appear.

Anti-Pattern 5: Deep Offset Pagination

Potentially slow:

```
SELECT *
FROM orders
ORDER BY order_date DESC
OFFSET 100000 ROWS FETCH NEXT 20 ROWS ONLY;
```

The database may still process a huge number of rows before returning 20.

Better for large datasets:

```
SELECT *
FROM orders
WHERE order_date < DATE '2026-06-01'
ORDER BY order_date DESC
FETCH NEXT 20 ROWS ONLY;
```

This is called keyset pagination.

8. Practical Query Tuning Checklist

When a query is slow, ask:

1. Are we filtering early?
2. Are join columns indexed?
3. Are filter columns indexed?
4. Are we selecting only needed columns?
5. Are we sorting a large result?
6. Are we using functions that block indexes?
7. Does the execution plan show a table scan?
8. Are row estimates very different from actual rows?
9. Is the query returning too much data for the application?
10. Would a different query shape be clearer or cheaper?

9. Mini Optimization Example

Slow pattern:

```
SELECT *
FROM orders
WHERE EXTRACT(YEAR FROM order_date) = 2026;
```

Problem:

The database has to apply a function to `order_date`, which may prevent normal index usage.

Better:

```sql
SELECT id, customer_id, order_date, total
FROM orders
WHERE order_date >= DATE '2026-01-01'
  AND order_date < DATE '2027-01-01';
```

This lets an index on `order_date` work naturally.

Practice Exercises

Exercise 1: Basic Join Practice

Create three tables:

```
customers(id, name, email)
orders(id, customer_id, order_date, total)
order_items(id, order_id, product_name, quantity, price)
```

Write queries to:

1. Show each customer with their orders.
2. Show each order with its order items.
3. Show customers who have never placed an order.
4. Show total spending per customer.
5. Show the top five customers by total order value.

Focus:

- `INNER JOIN`
- `LEFT JOIN`
- `GROUP BY`
- Aggregation

Exercise 2: Subquery vs Join

Write the same result two ways.

Task:

Find customers who placed an order over `$500`.

Subquery version:

```sql
SELECT name
FROM customers
WHERE id IN (
  SELECT customer_id
  FROM orders
  WHERE total > 500
);
```

Join version:

```sql
SELECT DISTINCT c.name
FROM customers c
JOIN orders o
  ON c.id = o.customer_id
WHERE o.total > 500;
```

Compare readability and execution plans.

Exercise 3: Add Indexes and Compare

Run a query before and after adding an index.

```sql
SELECT *
FROM orders
WHERE customer_id = 10;
```

Then add:

```sql
CREATE INDEX idx_orders_customer_id
ON orders(customer_id);
```

Run `EXPLAIN` before and after.

Focus:

- Seeing how indexes affect query plans
- Identifying index scans vs table scans

Exercise 4: Date Range Optimization

Write a query that finds all orders from 2026.

Avoid this:

```sql
WHERE EXTRACT(YEAR FROM order_date) = 2026;
```

Use this:

```sql
WHERE order_date >= DATE '2026-01-01'
  AND order_date < DATE '2027-01-01';
```

Then create an index:

```sql
CREATE INDEX idx_orders_order_date
ON orders(order_date);
```

Focus:

- Avoiding functions on indexed columns
- Writing index-friendly date filters

Exercise 5: Find Slow Query Patterns

Given these queries, identify what might be inefficient:

```sql
SELECT *
FROM orders;
```

```sql
SELECT *
FROM customers
WHERE LOWER(email) = 'test@example.com';
```

```sql
SELECT *
FROM products
WHERE name LIKE '%phone';
```

```sql
SELECT DISTINCT c.*
FROM customers c
JOIN orders o
  ON c.id = o.customer_id;
```

For each one, explain:

1. What is the problem?
2. Why can it be slow?
3. How would you improve it?

Exercise 6: Aggregation Practice

Write queries to answer:

1. Total revenue per month.
2. Average order value.
3. Number of orders per customer.
4. Products sold more than 10 times.
5. Customers with total spending greater than `$1000`.

Example:

```sql
SELECT customer_id, SUM(total) AS total_spent
FROM orders
GROUP BY customer_id
HAVING SUM(total) > 1000;
```

Exercise 7: Execution Plan Reading

For each query, run `EXPLAIN` or your database equivalent.

Look for:

- Table scan

- Index usage
- Join type
- Sort operation
- Estimated rows
- Filter condition

Then write a short explanation:

```
This query scans the orders table because there is no index on customer_id.
```

Exercise 8: Pagination Practice

Compare offset pagination:

```sql
SELECT *
FROM orders
ORDER BY order_date DESC
OFFSET 1000 ROWS FETCH NEXT 20 ROWS ONLY;
```

With keyset pagination:

```sql
SELECT *
FROM orders
WHERE order_date < DATE '2026-06-01'
ORDER BY order_date DESC
FETCH NEXT 20 ROWS ONLY;
```

Focus:

- Understanding why large offsets can become expensive
- Learning when keyset pagination is better

Exercise 9: Composite Index Practice

Create a query:

```sql
SELECT *
FROM orders
WHERE customer_id = 10
ORDER BY order_date DESC;
```

Then create:

```sql
CREATE INDEX idx_orders_customer_date
ON orders(customer_id, order_date);
```

Check whether the query plan improves.

Focus:

- Composite indexes

- Filtering and sorting with the same index

Exercise 10: Java and SQL Mini Practice

In a Java app using JDBC or Spring Data, write a repository or service method that retrieves:

1. Customer order history.
2. Customer total spending.
3. Recent orders by date.
4. Customers with no orders.

Then check the SQL being generated or executed.

Focus:

- Connecting database performance back to Java application code
- Avoiding hidden inefficient queries in application layers

Lab: Optimize Queries for an Order Management Database

Goal

Practice writing SQL queries, reading execution plans, adding indexes, and improving slow query patterns.

Database Tables

Use these tables:

```
CREATE TABLE customers (
    id INT PRIMARY KEY,
    name VARCHAR(100),
    email VARCHAR(150),
    city VARCHAR(100)
);

CREATE TABLE orders (
    id INT PRIMARY KEY,
    customer_id INT,
    order_date DATE,
    status VARCHAR(30),
    total DECIMAL(10, 2),
    FOREIGN KEY (customer_id) REFERENCES customers(id)
);

CREATE TABLE order_items (
    id INT PRIMARY KEY,
    order_id INT,
    product_name VARCHAR(100),
    quantity INT,
    price DECIMAL(10, 2),
    FOREIGN KEY (order_id) REFERENCES orders(id)
);
```

Sample Data

```
INSERT INTO customers VALUES
(1, 'Aisha Khan', 'aisha@example.com', 'Chicago'),
(2, 'Marcus Lee', 'marcus@example.com', 'Dallas'),
(3, 'Sofia Garcia', 'sofia@example.com', 'Phoenix'),
(4, 'Daniel Smith', 'daniel@example.com', 'New York'),
(5, 'Priya Patel', 'priya@example.com', 'Seattle');

INSERT INTO orders VALUES
(101, 1, DATE '2026-01-10', 'SHIPPED', 250.00),
(102, 1, DATE '2026-02-15', 'PENDING', 125.00),
(103, 2, DATE '2026-03-20', 'SHIPPED', 800.00),
(104, 3, DATE '2026-04-05', 'CANCELLED', 75.00),
(105, 4, DATE '2026-05-12', 'SHIPPED', 1200.00);

INSERT INTO order_items VALUES
(1, 101, 'Keyboard', 1, 100.00),
(2, 101, 'Mouse', 2, 75.00),
(3, 102, 'USB Cable', 5, 25.00),
(4, 103, 'Monitor', 2, 400.00),
(5, 104, 'Notebook', 3, 25.00),
(6, 105, 'Laptop', 1, 1200.00);
```

Part 1: Joins

Write a query to show each customer and their orders:

```
SELECT c.name, o.id AS order_id, o.order_date, o.total
FROM customers c
JOIN orders o
  ON c.id = o.customer_id;
```

Now write a query to show all customers, including customers with no orders:

```
SELECT c.name, o.id AS order_id, o.total
FROM customers c
LEFT JOIN orders o
  ON c.id = o.customer_id;
```

Question:

Which customer has no order?

Part 2: Aggregation

Find total spending by customer:

```
SELECT c.name, SUM(o.total) AS total_spent
FROM customers c
```

```
JOIN orders o
  ON c.id = o.customer_id
GROUP BY c.name;
```

Find customers who spent more than `$500`:

```
SELECT c.name, SUM(o.total) AS total_spent
FROM customers c
JOIN orders o
  ON c.id = o.customer_id
GROUP BY c.name
HAVING SUM(o.total) > 500;
```

Part 3: Subquery Practice

Find customers who placed an order above `$500` using a subquery:

```
SELECT name
FROM customers
WHERE id IN (
    SELECT customer_id
    FROM orders
    WHERE total > 500
);
```

Now rewrite it using a join:

```
SELECT DISTINCT c.name
FROM customers c
JOIN orders o
  ON c.id = o.customer_id
WHERE o.total > 500;
```

Question:

Which version do you find easier to read?

Part 4: Execution Plan

Run this query with `EXPLAIN`:

```
EXPLAIN
SELECT *
FROM orders
WHERE customer_id = 1;
```

Then create an index:

```
CREATE INDEX idx_orders_customer_id
```

```
ON orders(customer_id);
```

Run the plan again:

```
EXPLAIN
SELECT *
FROM orders
WHERE customer_id = 1;
```

Question:

Did the database use the index?

Part 5: Date Query Optimization

Bad pattern:

```
SELECT *
FROM orders
WHERE EXTRACT(YEAR FROM order_date) = 2026;
```

Better pattern:

```
SELECT *
FROM orders
WHERE order_date >= DATE '2026-01-01'
  AND order_date < DATE '2027-01-01';
```

Add an index:

```
CREATE INDEX idx_orders_order_date
ON orders(order_date);
```

Then run:

```
EXPLAIN
SELECT *
FROM orders
WHERE order_date >= DATE '2026-01-01'
  AND order_date < DATE '2027-01-01';
```

Part 6: Composite Index

Run this query:

```
EXPLAIN
SELECT *
FROM orders
```

```
WHERE customer_id = 1
ORDER BY order_date DESC;
```

Add a composite index:

```
CREATE INDEX idx_orders_customer_date
ON orders(customer_id, order_date);
```

Run the query again.

Question:

Why might this index be better than indexing only `customer_id`?

Part 7: Fix the Anti-Patterns

Rewrite these queries.

Bad:

```
SELECT *
FROM orders;
```

Better:

```
SELECT id, customer_id, order_date, total
FROM orders;
```

Bad:

```
SELECT *
FROM customers
WHERE LOWER(email) = 'aisha@example.com';
```

Better:

```
SELECT *
FROM customers
WHERE email = 'aisha@example.com';
```

Bad:

```
SELECT *
FROM orders
WHERE order_date + 1 = DATE '2026-01-11';
```

Better:

```sql
SELECT *
FROM orders
WHERE order_date = DATE '2026-01-10';
```

Deliverables

By the end of the lab, you should have:

1. Written join queries.
2. Written aggregation queries.
3. Compared subquery and join versions.
4. Used `EXPLAIN` .
5. Added single-column indexes.
6. Added a composite index.
7. Rewritten inefficient SQL patterns.

Challenge Task

Write one final report query.

Show each customer's:

- Name
- Number of orders
- Total spending
- Most recent order date

```sql
SELECT
    c.name,
    COUNT(o.id) AS order_count,
    COALESCE(SUM(o.total), 0) AS total_spent,
    MAX(o.order_date) AS most_recent_order
FROM customers c
LEFT JOIN orders o
  ON c.id = o.customer_id
GROUP BY c.name
ORDER BY total_spent DESC;
```

Key Takeaway

SQL performance is not about memorizing tricks. It is about reducing unnecessary work.

Good SQL makes the database do less:

- Fewer rows scanned
- Fewer columns returned
- Fewer unnecessary joins
- Fewer expensive sorts
- Better use of indexes